

DOCUMENTACIÓN DE DESARROLLO FRONTEND

RepairEquipment

Plataforma integral de gestión para un taller de reparación de equipos electrónicos. Este documento describe paso a paso cómo se desarrolló el frontend, su interacción con el backend y la base de datos, y las decisiones de diseño del proyecto.

React 19

Vite 7

Express 5

Node.js

MySQL 8

JWT Auth

RBAC 4 Roles

CONTENIDO

Tabla de Contenidos

- Arquitectura General del Proyecto
- Stack Tecnológico
- Estructura del Frontend
- Componentes y Vistas por Rol
- Flujo de Autenticación
- Comunicación Frontend ↔ Backend
- Referencia Completa de Rutas API
- Base de Datos — Esquema y Relaciones
- Flujo de Negocio Completo
- Dashboards por Rol — Detalle
- Formularios y Modales
- Sistema de Filtros y Búsqueda
- Módulo de Pagos y Facturación
- Documentación del Proyecto
- Instalación y Ejecución

SECCIÓN 01

Arquitectura General del Proyecto



Separación clara: El frontend no tiene acceso directo a la BD. Toda operación de datos pasa por la API REST, que valida autenticación (JWT) y autorización (roles) antes de ejecutar cualquier consulta SQL.

Estructura de Directorios

```
ProyectoReparacionEquipos/
├── backend/
│   ├── scripts/           # seedAdmin.js, seedDemo.js
│   └── src/
│       ├── controllers/   # Lógica de negocio por módulo (8 controllers)
│       ├── middleware/    # auth.js → JWT verify + role guards
│       ├── routes/        # Endpoints REST por dominio (8 routers)
│       ├── utils/         # http.js, text.js
│       ├── app.js         # Express app + montaje de rutas
│       ├── db.js          # Pool MySQL con mysql2/promise
│       └── server.js      # Arranque del servidor
├── frontend/
│   └── src/
│       ├── admin/
│       │   ├── views/
│       │   │   ├── AdminDashboard.jsx ← Componente raíz autenticado
│       │   │   └── sections/          ← 9 secciones del admin
│       │   └── components/
│       │       └── AdminFormsAndModals.jsx ← 18 forms/modales reutilizables
│       ├── auth/views/LoginView.jsx    ← Login + registro cliente
│       ├── cliente/views/ClienteDashboard.jsx
│       ├── recepcion/views/RecepcionDashboard.jsx
│       └── tecnico/views/TecnicoDashboard.jsx
└── docs/
    ├── schema.sql          # Esquema base de la BD
    ├── migrations/         # 4 migraciones incrementales
    ├── Database.md         # Documentación del modelo ER
    └── HistoriaUsuario.md  # 6 historias de usuario
```

SECCIÓN 02

Stack Tecnológico

Tecnologías y dependencias utilizadas en el proyecto.



Frontend

- **React 19.2** — Librería UI con hooks
- **Vite 7** — Bundler + dev server con HMR
- **CSS puro** — 1193 líneas en App.css
- Sin dependencias de UI externas (Tailwind, MUI, etc.)
- ESLint para linting del código



Backend

- **Express 5** — Framework HTTP
- **mysql2 3.16** — Driver MySQL con Promises
- **jsonwebtoken 9** — Generación/verificación JWT
- **bcryptjs 3** — Hash de contraseñas
- **cors** — Permitir peticiones cross-origin
- **dotenv** — Variables de entorno
- **nodemon** — Hot reload en desarrollo



Base de Datos

- **MySQL 8** — Motor InnoDB
- **13 tablas** — con relaciones FK
- **utf8mb4** — Soporte completo Unicode
- Compatible con XAMPP
- Migraciones incrementales SQL

SECCIÓN 03

Estructura del Frontend — Paso a Paso

Cómo se diseñó y construyó el frontend desde el punto de entrada hasta las vistas finales.

1

Punto de entrada: `main.jsx`

React se inicializa con `createRoot` montando el componente `<App />` dentro de `StrictMode` en el `div#root` del `index.html` . Se importa `index.css` para estilos globales base.

El `App.jsx` es mínimo, solo renderiza `<AdminDashboard />`. Este componente actúa como **Orquestador global** de toda la aplicación, manejando autenticación, estado global y enrutamiento por rol.

3 Decisión de vista por autenticación

El `AdminDashboard` verifica si existe `token` en `localStorage`. Si NO hay token → renderiza `<LoginView />`. Si hay token → verifica el rol del usuario y renderiza el dashboard correspondiente.

4 Enrutamiento basado en rol (sin React Router)

El proyecto **no usa React Router**. En lugar de eso, el `AdminDashboard` lee `user.rol` del estado y con condicionales renderiza el dashboard que corresponde:

`Técnico` → `<TecnicoDashboard />` · `Recepción` → `<RecepcionDashboard />` · `Cliente` → `<ClienteDashboard />` · `Administrador` → Panel admin con sidebar + secciones

5 Navegación interna con estado `view`

Dentro de cada dashboard, una variable de estado `const [view, setView] = useState('overview')` controla qué sección se muestra. El sidebar tiene botones `<button onClick={() ⇒ setView('ordenes')}>` que cambian la vista renderizada condicionalmente.

6 Carga de datos al autenticarse

Un `useEffect` ligado al `token` dispara la carga masiva de datos con `Promise.all()`: órdenes, clientes, equipos, técnicos, grupos, estados, tipos. Los datos extra (reglas, pagos, usuarios, roles) se cargan solo si el rol es Administrador o Recepción.

7 Estado centralizado con `useState` en AdminDashboard

Todo el estado de la aplicación (arrays de datos, filtros, modales abiertos, errores) se gestiona en `AdminDashboard` y se pasa vía **props** a las secciones y dashboards hijos. Esto convierte a `AdminDashboard` en un *state container* sin necesidad de Context API ni Redux.

```
// Flujo de renderizado simplificado:
main.jsx
├─ App.jsx
│   └─ AdminDashboard.jsx
│       ├── [sin token] → LoginView.jsx (login + registro cliente)
│       ├── [Técnico]   → TecnicoDashboard.jsx
│       ├── [Recepción] → RecepcionDashboard.jsx
│       ├── [Cliente]   → ClienteDashboard.jsx
│       └─ [Admin]     → Sidebar + Sections:
│           ├── OverviewSection.jsx
│           ├── OrdenesSection.jsx
│           ├── ClientesSection.jsx
│           ├── EquiposSection.jsx
│           ├── TecnicosSection.jsx
│           ├── GruposSection.jsx
│           ├── ReglasSection.jsx      (solo Admin)
│           ├── PagosSection.jsx       (solo Admin)
│           └─ UsuariosSection.jsx     (solo Admin)
```

Componentes y Vistas por Rol

Cada rol del sistema tiene un dashboard dedicado con funcionalidades específicas.

ADM

Administrador — AdminDashboard

Acceso completo al sistema con sidebar de 9 secciones:

- **Overview:** KPIs globales (totales, sin técnico, sin grupo, con saldo) + distribución por estado
- **Órdenes:** CRUD completo + filtros avanzados (estado, técnico, grupo, pendientes) + cambio rápido de estado y técnico inline
- **Clientes:** CRUD + búsqueda por nombre/documento/email
- **Equipos:** CRUD + asociación a cliente y tipo
- **Técnicos:** CRUD + toggle activo/inactivo
- **Grupos:** CRUD + asignar/remover técnicos de grupos + ver miembros
- **Reglas:** Reglas de asignación por palabras clave con prioridad
- **Pagos:** Historial filtrable con KPIs (total, hoy, promedio, cantidad) + exportar CSV
- **Usuarios:** Gestión de cuentas, roles, reset de password

TEC

Técnico — TecnicoDashboard

Vista enfocada en la carga de trabajo técnico:

- **KPIs:** Total asignadas, pendientes, en reparación, con diagnóstico
- **Mis órdenes:** Órdenes asignadas al técnico actual (match por email o nombre)
- **Todas las órdenes:** Vista lectura de todo el taller
- **Cambio rápido de estado:** Select inline para actualizar estado
- **Modal de trabajo:** Actualizar diagnóstico, observaciones y estado
- Filtros por estado y búsqueda textual

REC

Recepción — RecepcionDashboard

Punto de atención y registro:

- **Overview:** Contadores de órdenes, clientes, equipos + distribución por estado
- **Órdenes:** Crear órdenes + editar + cambiar estado/técnico + cobrar pagos
- **Clientes:** Registrar y editar clientes
- **Equipos:** Registrar y editar equipos con tipo y cliente
- **Pagos:** Historial + descarga de factura por orden
- Reutiliza formularios/modales del admin (AdminFormsAndModals)

CLI

Cliente — ClienteDashboard

Portal de autoservicio para el cliente:

- **KPIs:** Total órdenes, pendientes, listas, saldo total
- **Mi perfil:** Editar teléfono, dirección, ciudad
- **Mis órdenes:** Tabla con filtros por estado y búsqueda
- **Detalle de orden:** Modal con equipo, diagnóstico, historial, pagos
- **Pago en línea:** Simulación de tarjeta (validación de campos) + transferencia/efectivo/PSE
- **Factura:** Descarga de factura cuando el pago está completo

El sistema usa JWT (JSON Web Tokens) para autenticación sin estado. El frontend almacena el token y datos del usuario en localStorage.

1 Login — POST /api/auth/login

El frontend envía `{ email, password }`. El backend verifica credenciales con `bcryptjs` y retorna `{ token, user: { id, nombre, apellido, email, rol } }`.

2 Almacenamiento del Token

El frontend guarda `token` y `user` (JSON serializado) en `localStorage`. Al recargar la página, `useState` recupera estos valores con funciones inicializadoras:

```
const [token, setToken] = useState(() => localStorage.getItem('token') || '')
const [user, setUser] = useState(() => {
  const raw = localStorage.getItem('user')
  return raw ? JSON.parse(raw) : null
})
```

3 Envío del Token en cada Petición

Cada llamada a la API incluye el header `Authorization: Bearer {token}`. Las funciones helper `apiGet`, `apiPost`, `apiPut`, `apiDelete` centralizan esta lógica.

4 Verificación en el Backend (Middleware)

El middleware `authRequired` extrae el token del header, lo verifica con `jwt.verify()` y adjunta `req.user` (payload del JWT) al request. Si falla → 401. Luego `requireRole('Administrador')` o `requireAnyRole([...])` verifican permisos → 403 si no coincide.

5 Registro de Cliente — POST /api/auth/register-cliente

Desde el `LoginView`, el formulario de registro envía nombre, apellido, documento, teléfono, email, dirección, ciudad y password. El backend crea simultáneamente un registro en `clientes` y otro en `usuarios` con rol "Cliente".

6 Logout

El frontend limpia `localStorage` y resetea los estados `token` y `user` a vacío/null. Esto provoca que el componente vuelva a renderizar el `LoginView`.

Comunicación Frontend ↔ Backend

El frontend se comunica con el backend exclusivamente mediante la API Fetch nativa del navegador. No se usa Axios ni librerías HTTP externas.

Dentro de `AdminDashboard.jsx` se definen 4 funciones que encapsulan las operaciones HTTP con token JWT:

```
// GET con autenticación
const apiGet = async (path) => {
  const res = await fetch(`http://localhost:3001${path}`, {
    headers: { Authorization: `Bearer ${token}` },
  })
  if (!res.ok) throw new Error('Error al cargar datos')
  return res.json()
}

// POST con autenticación + body JSON
const apiPost = async (path, payload) => {
  const res = await fetch(`http://localhost:3001${path}`, {
    method: 'POST',
    headers: {
      Authorization: `Bearer ${token}`,
      'Content-Type': 'application/json',
    },
    body: JSON.stringify(payload),
  })
  const data = await res.json()
  if (!res.ok) throw new Error(data?.error || 'Error al guardar')
  return data
}

// PUT y DELETE siguen el mismo patrón...
```

Patrón de Actualización: Operación + Refresh

Después de cada operación de escritura (crear, editar, eliminar), el frontend refresca los datos correspondientes desde la API:

```
const createOrden = async (payload) => {
  await apiPost('/api/ordenes', payload) // 1. Enviar al backend
  await refreshOrdenes()                 // 2. Recargar la lista
}

// refreshOrdenes() simplemente hace:
const refreshOrdenes = async () => setOrdenes(await apiGet('/api/ordenes'))
```

Ciclo de vida de un dato: Frontend (estado React) → fetch POST/PUT/DELETE → Backend (controller valida + ejecuta SQL) → Response JSON → Frontend dispara GET de refresh → setState actualiza la UI.

Carga Inicial Masiva con Promise.all

Al autenticarse, un `useEffect` carga todos los datos necesarios en paralelo:

```
useEffect(() => {
  if (!token) return
  const load = async () => {
    const basePromises = [
      apiGet('/api/ordenes'),      apiGet('/api/clientes'),
      apiGet('/api/equipos'),      apiGet('/api/tecnicos'),
      apiGet('/api/grupos'),       apiGet('/api/estados'),
    ]
```

```
// Extras solo para Admin o Recepción:
const extraPromises = isAdmin
  ? [apiGet('/api/reglas-asignacion'), apiGet('/api/pagos'),
    apiGet('/api/usuarios'), apiGet('/api/usuarios/roles')]
  : isRecepcion
  ? [Promise.resolve([]), apiGet('/api/pagos'), ...]
  : [Promise.resolve([]), ...]

const results = await Promise.all([...basePromises, ...extraPromises])
// Desestructurar y setear cada estado...
}
load()
}, [token, user?.rol])
```

Referencia Completa de Rutas API

Todos los endpoints REST expuestos por el backend en `http://localhost:3001/api`. Los roles indicados son los que tienen acceso a cada ruta.

Autenticación

MÉTODO	RUTA	DESCRIPCIÓN	ROLES
POST	<code>/auth/login</code>	Iniciar sesión → retorna JWT + user	Público
POST	<code>/auth/register-cliente</code>	Crear cuenta de cliente	Público

Catálogos

MÉTODO	RUTA	DESCRIPCIÓN	ROLES
GET	<code>/estados</code>	Lista estados de orden	Autenticado
GET	<code>/tipos-equipo</code>	Lista tipos de equipo	Autenticado

Órdenes de Servicio

MÉTODO	RUTA	DESCRIPCIÓN	ROLES
GET	<code>/ordenes</code>	Listar todas las órdenes	Admin, Técnico, Recepción
GET	<code>/ordenes/:id</code>	Detalle de orden + historial	Admin, Técnico, Recepción
POST	<code>/ordenes</code>	Crear nueva orden	Admin, Técnico, Recepción
PUT	<code>/ordenes/:id</code>	Actualizar orden completa	Admin, Técnico, Recepción
PATCH	<code>/ordenes/:id/estado</code>	Cambiar solo el estado	Admin, Técnico, Recepción

POST	/ordenes/:id/asignar	Asignar técnico/grupo manual	Admin
GET	/ordenes/:id/sugerencias	Sugerencias por palabras clave	Admin
POST	/ordenes/:id/asignar-sugerido	Asignar técnico/grupo sugerido	Admin

Portal Cliente

MÉTODO	RUTA	DESCRIPCIÓN	ROLES
GET	/mis-ordenes	Órdenes del cliente autenticado	Cliente
GET	/mis-ordenes/:id	Detalle de orden propia	Cliente
GET	/mis-ordenes/:id/factura	Descargar factura propia	Cliente
GET	/clientes/me	Datos del perfil propio	Cliente
PUT	/clientes/me	Actualizar perfil propio	Cliente

Pagos y Facturas

MÉTODO	RUTA	DESCRIPCIÓN	ROLES
POST	/ordenes/:id/pagos	Registrar pago	Autenticado
GET	/ordenes/:id/pagos	Pagos de una orden	Autenticado
GET	/pagos	Todos los pagos	Admin, Recepción
GET	/ordenes/:id/factura	Descargar factura (staff)	Admin, Recepción

Cientes, Equipos, Técnicos, Grupos, Reglas, Usuarios

RECURSO	GET LIST	GET :ID	POST	PUT :ID	DELETE :ID
Cientes	✓	✓	✓	✓	Admin
Equipos	✓	✓	✓	✓	Admin
Técnicos	✓	—	✓	✓	Admin
Grupos	✓	técnicos	Admin	Admin	Admin
Reglas	Admin	—	Admin	Admin	Admin
Usuarios	Admin	—	Admin	Admin	Admin

MySQL 8 con 13 tablas InnoDB, integridad referencial completa con foreign keys, y migraciones incrementales.

R

roles

4 roles: Administrador, Cliente, Tecnico, Recepcion.
Referenciado por `usuarios.rol_id`.

U

usuarios

Cuentas de login. FK a `roles` y opcionalmente a `clientes`. Almacena password_hash (bcrypt).

C

clientes

Propietarios de equipos. Datos personales + documento único. Referenciado por equipos, órdenes, pagos.

T

tecnicos

Personal técnico con especialidad y estado activo/inactivo. Relacionado con grupos vía `tecnico_grupos`.

G

grupos_responsables

Áreas o equipos de trabajo. Relación N:M con técnicos. Referenciado por órdenes y reglas.

TG

tecnico_grupos

Tabla pivote N:M entre técnicos y grupos. PK compuesta (`tecnico_id`, `grupo_id`).

RA

reglas_asignacion

Palabras clave separadas por coma → asociadas a un grupo y opcionalmente un técnico, con prioridad.

TE

tipos_equipo

Catálogo: PC, Laptop, Impresora, Monitor, Tablet, Celular.

E

equipos

Equipos de clientes con marca, modelo, serie (única), tipo y accesorios.

EO

estados_orden

Catálogo: Recibido → En Diagnóstico → En Reparación → Listo → Entregado.

OS

ordenes_servicio

Tabla central. FK a cliente, equipo, técnico, grupo, estado. Campos de costo, saldo, pagado, garantía, falla, diagnóstico.

OH

ordenes_historial

Log de cambios de estado por orden. Registra estado, comentario, usuario y fecha.

pagos

Pagos asociados a órdenes. Método, valor, referencia, cliente y usuario que registró.

Relaciones Clave (ER)

- roles —1:N→ usuarios
- clientes —1:N→ usuarios (opcional, solo rol Cliente)
- clientes —1:N→ equipos
- clientes —1:N→ ordenes_servicio
- clientes —1:N→ pagos
- tipos_equipo —1:N→ equipos
- equipos —1:N→ ordenes_servicio
- tecnicos —1:N→ ordenes_servicio
- tecnicos —N:M→ grupos_responsables (vía tecnico_grupos)
- tecnicos —1:N→ reglas_asignacion
- grupos —1:N→ ordenes_servicio
- grupos —1:N→ reglas_asignacion
- estados_orden —1:N→ ordenes_servicio
- estados_orden —1:N→ ordenes_historial
- ordenes —1:N→ ordenes_historial
- ordenes —1:N→ pagos
- usuarios —1:N→ ordenes_historial
- usuarios —1:N→ pagos

SECCIÓN 09

Flujo de Negocio Completo

El ciclo de vida de una orden de servicio desde la recepción hasta la entrega y facturación.

1 Recepción registra cliente y equipo

Desde RecepcionDashboard o AdminDashboard, se crea el cliente (si es nuevo) y luego su equipo con marca, modelo, serie y tipo. El frontend envía `POST /api/clientes` y `POST /api/equipos` .

2 Crear orden de servicio

Se crea la orden con código (ej: OS-0001), cliente, equipo, falla reportada y costo estimado. El backend registra estado "Recibido" y crea entrada en `ordenes_historial` .

3 Admin asigna técnico y grupo

Manualmente o usando sugerencias por palabras clave (reglas de asignación). El frontend permite cambio inline de técnico/grupo desde la tabla de órdenes, o mediante el modal de edición completo.

4 Técnico actualiza diagnóstico y estado

5

Orden llega a estado "Listo"

Se habilita el pago. Tanto el portal del cliente como recepción pueden registrar pagos. El `costo_final` define el saldo pendiente.

6

Registro de pago

Cliente: paga desde su portal con tarjeta (simulada), transferencia, PSE o efectivo. Recepción: registra pago en mostrador. El backend actualiza `saldo` y marca `pagado = 1` cuando saldo llega a 0.

7

Generación y descarga de factura

Con la orden pagada, se habilita descarga de factura (PDF/HTML generado por el backend). Accesible desde el portal del cliente y desde recepción.

SECCIÓN 10

Dashboards por Rol — Detalle Técnico

AdminDashboard — State Container

Es el corazón de la aplicación. Gestiona **+30 variables de estado** con `useState` : datos de todas las entidades, filtros, modales abiertos, errores, loading. Define las funciones CRUD que se pasan como props a las secciones hijas.

También calcula datos derivados con `useMemo` : conteos por estado, filtros combinados de órdenes/clientes/equipos/pagos, estadísticas de pagos.

TecnicoDashboard — Match Inteligente

El técnico no tiene un `tecnico_id` directo en su JWT. El dashboard hace **match inteligente**: busca en la lista de técnicos aquel cuyo email o nombre+apellido coincida con el usuario logueado. Luego filtra `ordenes` por ese tecnico_id.

ClienteDashboard — Autónomo

A diferencia de los otros dashboards, el ClienteDashboard **gestiona su propio estado**. No recibe datos del AdminDashboard padre; tiene sus propias funciones `apiGet` , `apiPost` , `apiPut` internas y consume endpoints exclusivos: `/mis-ordenes` , `/clientes/me` .

RecepcionDashboard — Reutilización

Recibe todos los datos y funciones CRUD desde AdminDashboard vía props. Reutiliza componentes de `AdminFormsAndModals` (OrdenCreateForm, ClienteForm, EquipoForm, modales de edición). Añade su propio `RecepcionPagoModal` para cobros en mostrador.

SECCIÓN 11

Formularios y Modales

Formularios Inline

- GrupoForm
- TecnicoForm
- ClienteForm
- EquipoForm
- ReglaForm
- UsuarioForm
- OrdenCreateForm
- GrupoAssignForm

Se renderizan directamente dentro de la sección. Cada uno maneja su estado local y llama `onCreate` (prop) al enviar.

Modales de Edición

- OrdenModal
- ClienteModal
- EquipoModal
- TecnicoModal
- GrupoModal
- ReglaModal
- UsuarioModal
- UsuarioPasswordModal

Se abren al hacer click en "Editar". Overlay con `modal-backdrop` + formulario precargado con datos actuales.

Modales de Detalle

- OrdenDetalleModal
- GrupoTecnicosModal
- ClienteDetalleModal (en ClienteDashboard)
- PagoModal (en ClienteDashboard)
- TecnicoOrdenModal (en TecnicoDashboard)
- RecepcionPagoModal

Consultan datos adicionales al abrirse (ej: historial, pagos) y presentan información de solo lectura o acciones específicas.

Patrón modal: El estado `editOrden` (null o objeto) controla la visibilidad. Cuando no es null, se renderiza el modal. Al cerrar: `setEditOrden(null)` . Al guardar: `apiPut` → refresh → `setEditOrden(null)` .

SECCIÓN 12

Sistema de Filtros y Búsqueda

Cada sección implementa filtrado en el cliente usando `useMemo` para recalcular listas filtradas reactivamente.

Filtros de Órdenes (Admin)

- Búsqueda textual:** código, nombre cliente, marca, modelo, serie
- Estado:** select con todos los estados
- Técnico:** select con todos los técnicos
- Grupo:** select con todos los grupos

Filtros de Pagos (Admin)

- Búsqueda:** orden, cliente, referencia
- Método de pago:** dinámico según métodos existentes
- Rango de fechas:** desde/hasta con inputs tipo date

- **KPIs reactivos:** total, total hoy, promedio ticket, cantidad — se recalculan según filtros

```
// Ejemplo: filtrado de órdenes con useMemo
const filteredOrdenes = useMemo(() => {
  const q = ordenSearch.trim().toLowerCase()
  return ordenes.filter((o) => {
    if (ordenEstado && String(o.estado_id) !== String(ordenEstado)) return false
    if (ordenTecnico && String(o.tecnico_id) !== String(ordenTecnico)) return false
    if (ordenGrupo && String(o.grupo_id) !== String(ordenGrupo)) return false
    if (ordenSoloPendientes && o.estado_nombre === 'Entregado') return false
    if (!q) return true
    return String(o.codigo).toLowerCase().includes(q) ||
      String(o.cliente_nombre).toLowerCase().includes(q) || ...
  })
}, [ordenes, ordenSearch, ordenEstado, ordenTecnico, ordenGrupo, ordenSoloPendientes])
```

SECCIÓN 13

Módulo de Pagos y Facturación

El sistema soporta pagos desde dos puntos: el portal del cliente y la recepción del taller.

Pago desde Portal Cliente

El `PagoModal` ofrece métodos: Transferencia, Tarjeta, Efectivo, PSE. Para tarjeta, incluye validación simulada (número 12-19 dígitos, titular, MM/AA, CVV). La referencia se genera automáticamente con los últimos 4 dígitos.

Se habilita cuando estado = "Listo" y `pagado = false`. No permite valores superiores al saldo pendiente.

Pago desde Recepción

El `RecepcionPagoModal` es más simple: método (Efectivo/Tarjeta/Transferencia), valor y referencia opcional. Se muestra el botón "Cobrar" en órdenes con estado Listo/Entregado que tengan saldo pendiente.

Recepción también puede descargar facturas desde el historial de pagos para cualquier orden.

Flujo del pago en el backend: El controller `createPago` inserta en la tabla `pagos`, luego recalcula el saldo de la orden sumando todos los pagos existentes y actualizando `ordenes_servicio.saldo` y `ordenes_servicio.pagado`.

SECCIÓN 14

Documentación del Proyecto

El proyecto incluye documentación técnica y funcional en la carpeta `docs/`.

RE::DOCS

DB

Arquitectura

Tech
Stack

Frontend

Componentes

API

Base de
Datos

Flujo

Documentación

Database.md

Documentación completa del modelo ER en formato Mermaid. Describe cada tabla, su propósito y todas las relaciones con claves foráneas. Incluye notas sobre restricciones de eliminación.

SQL

schema.sql + migrations/

Esquema base con 13 tablas + datos semilla (roles, estados, tipos). 4 migraciones incrementales para grupos, pagos de cliente y reglas de asignación.

UML

DiagramaClases.puml + Database.puml

Diagramas PlantUML del modelo de clases y del esquema de base de datos para visualización técnica profesional.

ST

Estructura.md

Documento con el árbol de directorios del proyecto y notas sobre la organización modular del código.

MK

MockupsLineaEnfasis.pdf

Mockups visuales de la línea de énfasis del proyecto con diseños de las interfaces planeadas.

Historias de Usuario — Resumen

ID	TÍTULO	ACTOR	DESCRIPCIÓN
HU-01	Registrar cliente, equipo y orden	Recepción / Admin	Iniciar flujo de reparación con trazabilidad desde recepción
HU-02	Gestionar ciclo técnico	Técnico	Actualizar estado, diagnóstico y observaciones de órdenes asignadas
HU-03	Grupos y reglas de asignación	Admin	Crear grupos, asignar técnicos, definir reglas por palabras clave
HU-04	Gestión de usuarios y seguridad	Admin	CRUD usuarios, roles, reset password, sincronización con clientes/técnicos
HU-05	Registro de pago en punto de atención	Recepción / Admin	Registrar pagos en mostrador para cerrar saldo de órdenes
HU-06	Portal cliente: consulta y pago	Cliente	Ver estado de órdenes, pagar en línea, descargar factura

```
mysql -u root -p < docs/schema.sql
mysql -u root -p TallerBD < docs/migrations/2026-02-07_add_client_pagos.sql
mysql -u root -p TallerBD < docs/migrations/2026-02-07_add_grupos.sql
mysql -u root -p TallerBD < docs/migrations/2026-02-07_add_reglas_asignacion.sql
mysql -u root -p TallerBD < docs/migrations/2026-02-07_update_reglas_asignacion_palabras.sql
```

2 Configurar variables de entorno

Crear archivo `backend/.env` :

```
PORT=3001
DB_HOST=127.0.0.1
DB_USER=root
DB_PASS=
DB_NAME=TallerBD
DB_PORT=3306
JWT_SECRET=change_me_please
ADMIN_EMAIL=admin@taller.local
ADMIN_PASSWORD=Admin1234
ADMIN_NOMBRE=Admin
ADMIN_APELLIDO=Principal
```

3 Instalar dependencias e iniciar

```
# Backend
cd backend && npm install
npm run seed:admin      # Crear usuario administrador
npm run seed:demo       # Datos de demostración
npm run dev              # Inicia en :3001

# Frontend
cd frontend && npm install
npm run dev              # Inicia en :5173
```

4 Acceso al sistema

Abrir `http://localhost:5173` . Credenciales de demo:

Admin: `admin@taller.local` / `Admin1234`

Cliente demo: `carlos@mail.com` / `Cliente1234`